DECISION PIPELINE REFERENCE

# Jev vs. LLM vs. Rules: Fraud Decision Pipeline

Built for **fraud-flag**, a streaming payment-fraud triage service
Rules decide the obvious cases, Jev the ambiguous ones, Claude only writes case summaries

**PARTIALLY MOCK:** `DecisionClient` is still `MockDecisionClient` -- no documented request/response shape for the real Jev (TypeSafe) `noul/choice` API exists yet for this project, per CLAUDE.md's "don't invent SDK calls" rule. `CaseSummaryClient` is the real Anthropic API (Claude Haiku 4.5) -- wired and used in the run below. Both panels are real output from this repo, just split by which decision-maker actually handled each case.

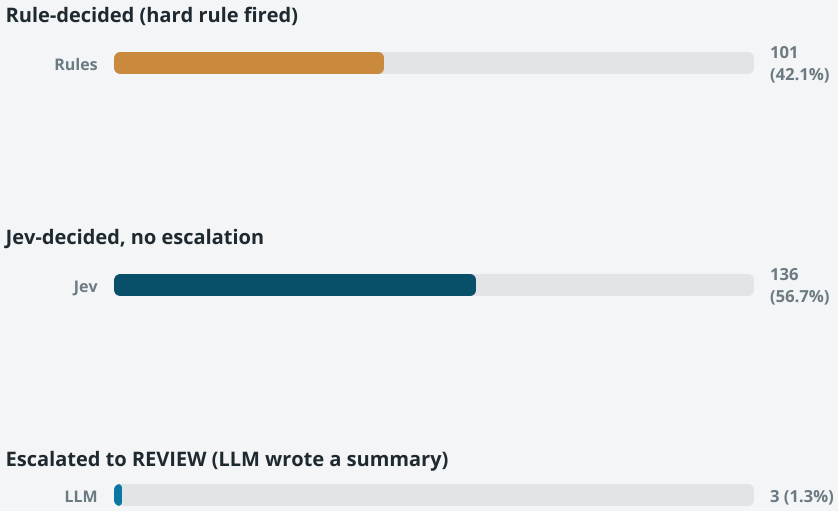
**Rules Engine**
Hard-coded checks for obvious cases (RulesEngine.java)

**Jev (mock DecisionClient)**
Bounded noul()/choice() calls for whatever a rule didn't resolve

**LLM: Claude Haiku 4.5 (real)**
Writes a case summary for REVIEW cases -- never decides

## Decision volume by path

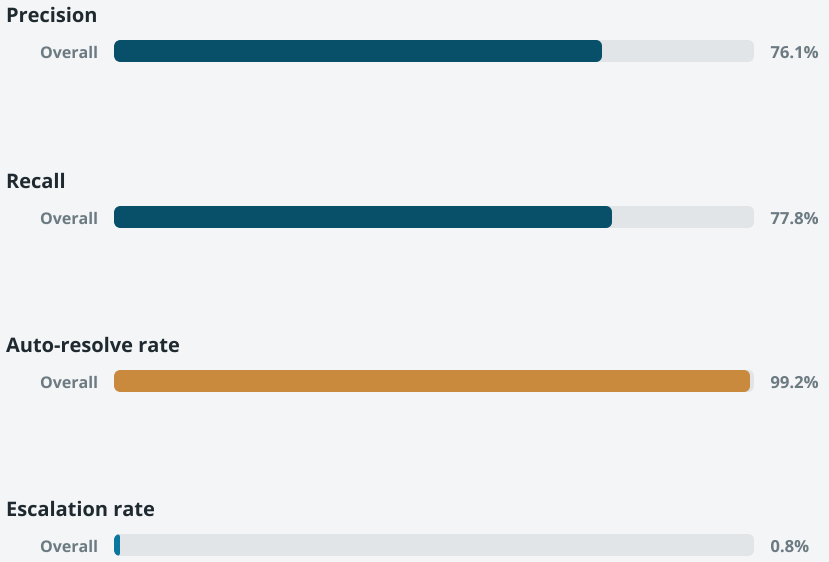
n=240 decisions, this repo's accumulated local runs (decisions table)




*Rules and Jev together resolve ~99% of traffic without ever reaching the LLM -- that's by design, see page 3.*

## Pipeline accuracy vs. ground truth

n=120 labeled transactions, one generator run, scripts/score.py




*These are pipeline-wide (rules + mock Jev combined) -- ground truth doesn't label which component made each call.*

Source: fraud-flag (this repo), Postgres `decisions` table and scripts/score.py. Real data from local runs, not benchmarked/published figures.

DECISION PIPELINE REFERENCE

# Pros & Cons by Approach

## Rules Engine

RulesEngine.java

### PROS

- Zero latency: 0.07ms average in this run, no network call at all
- Fully deterministic and explainable -- the exact condition is visible in the diff
- No API cost, no dependency on an external service being up
- Catches the fraud patterns the generator itself injects, by construction

### CONS

- Only as good as the rules someone wrote -- a new pattern needs a new rule
- Binary: no notion of "a little suspicious" vs. "extremely suspicious"
- Easy to write a rule that's too broad and silently swallows edge cases

### BEST FIT

The small number of fraud patterns with an unambiguous signature -- a burst of tiny card-testing charges, an amount 10x the account's usual spend. Exactly Phase 3's "obvious cases" design intent.

## Jev

DecisionClient / MockDecisionClient

### PROS

- Structurally bounded: noul() always returns boolean+confidence, choice() always returns one of the given fraud types -- can't answer outside that shape
- Fast enough to run on every transaction that reaches it (136/240 here, <0.1ms avg)
- The interface seam means a real Jev client swaps in later with zero other code changes

### CONS

- Still MockDecisionClient here -- no real Jev/TypeSafe calibration data exists for this project yet, unlike confidence-audit's project where a real smoke test already ran
- Mock confidence is a simple suspicious-category count, not a trained/calibrated score

### BEST FIT

The ambiguous middle: cases a rule doesn't confidently resolve but still need a fast, bounded fraud/not-fraud call to decide routing.

## LLM (Claude Haiku 4.5)

CaseSummaryClient / AnthropicCaseSummaryClient

### PROS

- The only one of the three that writes prose a human reviewer can actually read
- Real and wired, not mocked -- actually called Claude Haiku 4.5 in the run behind this doc
- Never asked to decide, only to explain -- its failure mode can't silently flip a route

### CONS

- By far the slowest path measured: 8.5s average, 23s max in this run, vs. sub-millisecond for the other two -- only viable gated to ~1% of traffic
- Costs money per call, unlike the other two
- One of three live calls in this run returned an empty response with no error -- needs a retry/fallback that doesn't exist yet

### BEST FIT

Exactly the ~1% of cases that reach REVIEW -- never routing itself, only making the human reviewer's job faster once a transaction is already flagged as ambiguous.

Sources: this repo's src/main/java/com/fraudflag/decision/*.java and the same local run cited on page 1.

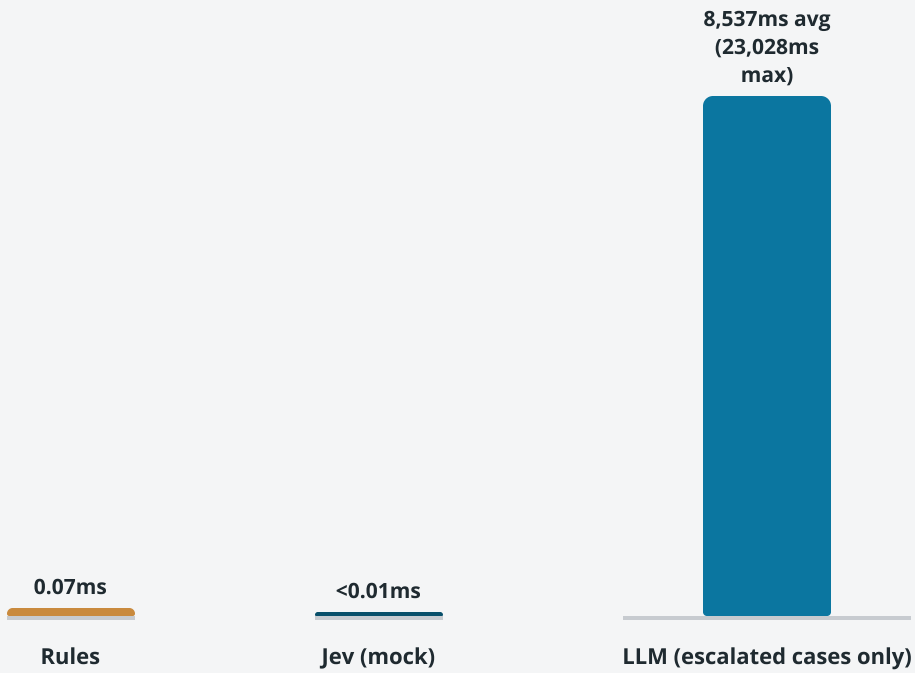
DECISION PIPELINE REFERENCE

# Why Rules First, Jev Fallback, LLM Last?

The pipeline is layered cheapest-and-fastest-first. A **hard rule** runs first and, when it fires, decides the route for free in well under a millisecond -- no network call, no cost, fully explainable. Anything a rule doesn't confidently resolve falls to **Jev**, still bounded and still fast, trading a little certainty for coverage of the ambiguous middle. Only the rarest already-escalated case -- about 1% of traffic in this run -- ever pays the latency and dollar cost of a real LLM call, and even then the LLM **never decides**: it only writes a case summary for the human who does. This is CLAUDE.md's explicit design, not something improvised after the fact: "the LLM is called only for uncertain cases… and never makes the decision."

## Decision latency by path (log-scaled)

Same n=240 run as page 1 -- bar heights are log-scaled; the gap is the point




*Rules and Jev bars are near-invisible on purpose: they really are ~100,000x faster than a live LLM call. That gap is exactly why escalation is capped near 1% instead of routing every transaction through the LLM.*

Source: fraud-flag (this repo), Postgres `decisions` table, this session's local runs. Next: real Jev client once TypeSafe docs exist; retry/fallback for empty LLM responses.